
HIGH PERFORMANCE PROGRAMMING

WORKSHOP

EDITED BY

GUSTAV NYBRO
NIKOLAJ ALBERTSEN
MAGNUS VØRS HOLMSGÅRD
VIKTOR KIRK ALMANN HANSEN

THE UNIVERSITY OF AALBORG



2026

GROUP 630

Contents

1 Introduction	2
Workshop description	2
Instruction	2
1: Breadth-First Search (BFS) for Large Graphs:	3
General	3
Problem A	3
1.1 Answer: A (I)	4
1.2 Answer: A (II)	4
1.3 Answer: A (III)	5
1.4 Answer: A (IV)	5
Problem B	7
1.5 Answer: B (I)	7
1.6 Answer: B (II)	8
1.7 Answer: B (III)	9
1.8 Answer: B (IV)	9
1.9 Answer: B (V)	10
Problem C	12
1.10 Answer: C (I)	12
1.11 Answer: C (II)	13
1.12 Answer: C (III)	13
1.13 Answer: C (IV)	14
2: The JPEG image compression algorithm:	16
General	16
Problem	16
Methods to reduce execution time covered in the course.	17
1.14 Problem: (I)	18
1.15 Problem: (II)	18
1.16 Problem: (III)	19
1.17 Problem: (IV)	20
1.18 Problem: (V)	22
1.19 Problem: (VI)	25
1.20 Problem: (VII)	26

1 Introduction

Workshop description

The workshop exercises cover the design and analysis of parallel algorithms as well as parallel programming on shared and distributed memory architectures. For each problem, you are expected to provide the entire development pipeline for parallel programs. The exercises should be solved in groups.

The workshop session will take place on May 7, 2026. Here are the instructions to conduct the workshop exercises.

Instruction

Before the workshop: Solve the problems below, starting with the design of a solution for them. That is, try to specify how you'll solve the problem before you start coding. You'll notice that you have to make some critical choices for their implementation. These choices must be documented, for example, whether you used vectorization, multithreading or multiprocessing, MPI, pipelining or even GPU programming. Make sure that you also implement the code to obtain the average execution time of your algorithms, executing them several times. Important: You'll need to submit your codes and the necessary files to run them in Moodle before the workshop.

During the workshop: We'll meet in the lecture room and discuss the implementation choices. We'll also try to compare the execution time of the solutions that you implemented.

After the workshop: A single final report per group should be handed in by May 15, 2026.

1: Breadth-First Search (BFS) for Large Graphs

General

Breadth-First Search (BFS) on graphs is a key building block in many real-world applications, including web crawling, social network analysis, recommendation systems, shortest-path algorithms, and more. The focus of this problem is on - designing a sequential BFS algorithm for very large graphs (represented in either adjacency matrix or adjacency list form, depending on memory constraints); analysing the work and depth of this BFS; developing a parallel version for shared memory (e.g., multi-core CPU); and extending the approach for distributed memory architectures (e.g., MPI or a cluster). You will implement your solutions in code, measure performance, and compare to the theoretical predictions about speedup, efficiency, communication overhead, and so on.

```
1 procedure BFS(G,s):
2   Create a queue Q
3   for each vertex v in G:
4     distance[v] =  $\infty$ 
5     visited[v] = false
6   distance[s] = 0
7   visited[s] = true
8   enqueue(s,Q)
9
10  while Q is not empty:
11    u = dequeue(Q)
12    for each neighbour w of u in G:
13      if not visited[w]:
14        visited[w] = true
15        distance[w] = distance[u] + 1
16        enqueue(w, Q)
```

Algorithm 1: Sequential BFS

Problem A

Let G be a graph with V vertices and E edges. You need to explore G starting from a specific source vertex s , visiting all reachable vertices in order of increasing distance from s .

- (I) Explain why the total work done by your BFS procedure is $\Theta(V + E)$. Discuss the "depth" of BFS in a parallel sense, focusing on the number of BFS levels (which often equals the graph's diameter or a bound on it).
- (II) Implement the sequential queue-based BFS (see Algorithm 1) in a language of your choice. Clearly document how you generate the graph and how you measure the elapsed time.
- (III) Create two graphs, G_1 and G_2 , with G_2 having at least a factor of 10 larger number of nodes than G_1 . You should try to make G_1 as large as possible.
- (IV) Discuss whether BFS can be considered "embarrassingly parallel" or not. Briefly justify your reasoning in terms of data dependencies and communication.

1.1 Answer: A (I)

Explain why the total work done by your BFS procedure is $\Theta(V + E)$.

BFS visits each vertex exactly once. When we mark a node as visited, we never process it again, resulting in $\Theta(V)$. For each vertex, we look at all its neighbors, this means we go through all adjacency lists, which results in $\Theta(E)$. For a directed graph we look at all edges once and for undirected graphs we look at edges twice. These are both constant which is why $\Theta(E)$. The total work is then the combined sum, $\Theta(V + E)$.

Discuss the "depth" of BFS in a parallel sense, focusing on the number of BFS levels (which often equals the graph's diameter or a bound on it).

When looking at parallel computing: We don't care about total work, instead we care about how many sequential steps are required.

A way of thinking of it: BFS works in levels: Where; level 0 $\rightarrow$ start node, Level 1 $\rightarrow$ neighbors, Level 2 $\rightarrow$ neighbors of neighbors, etc.

When working in parallel, nodes in the same level can be processed in parallel, but the level "k" needs to be finished before level "k+1". Therefore depth is the number of BFS levels, which is equal to the maximum distance from the source node to any reachable node. This is often bounded by the diameter of the graph, proportional to the number of BFS levels, often written as: $\Theta(D)$.

comment:

1.2 Answer: A (II)

Implement the sequential queue-based BFS (see Algorithm 1) in a language of your choice.

We picked python as the language of choice. The code can be found on github: HPP_630_COMTEK

Clearly document how you generate the graph and how you measure the elapsed time.

The graph is generated using the Barabási–Albert model, which produces a scale-free network. Resulting in some nodes becoming highly connected (hubs), while most nodes have few connections.

The graph is represented as an adjacency list, where each node stores a list of its neighbors:

```
17 adj = {v: [] for v in range(n)}
```

Construction begins with a fully connected core of $m + 1$ nodes:

```
18 for u in range(m + 1):
19     for v in range(u + 1, m + 1):
20         adj[u].append(v)
21         adj[v].append(u)
22     stubs += [u, v]
```

New nodes are then added one by one. Each new node connects to "m" existing nodes, where the probability of selecting a node is proportional to its degree (number of neighbors it already have). This is implemented using the list "stubs", where nodes appear multiple times based on how many edges they have:

```
23 for v in range(m + 1, n):
24     targets = set()
25     while len(targets) < m:
```

```
26     targets.add(rng.choice(stubs))
```

The selected targets are then connected to the new node:

```
27 for t in targets:
28     adj[v].append(t)
29     adj[t].append(v)
30     stubs += [v, t]
```

The execution time of the BFS algorithm is measured using repeated runs and averaging the results.

```
31 for _ in range(runs):
```

First a timestamp is recorded before execution.

```
32 t0 = time.perf_counter()
```

Executes the BFS function with the given inputs.

```
33 fn(*args)
```

Computes the time the function took to execute, and saves the measured runtime.

```
34 ts.append(time.perf_counter() - t0)
```

Returns the mean execution time across all runs.

```
35 return sum(ts) / len(ts)
```

1.3 Answer: A (III)

Create two graphs, G_1 and G_2 , with G_2 having at least a factor of 10 larger number of nodes than G_1 . You should try to make G_1 as large as possible.

The number of edges was scaled proportionally to maintain similar graph density

The first graph G_1 contains 50,000 vertices, and 249,985; Average sequential BFS time: 42.1 milliseconds

The second graph G_2 contains 500,000 vertices, and 2,499,985 edges; Average sequential BFS time: 1557.5 milliseconds

(disclaimer: the average sequential BFS time varies greatly depending on hardware, but the numbers stated above are results from a sequential run under similar conditions.)

1.4 Answer: A (IV)

Discuss whether BFS can be considered "embarrassingly parallel" or not. Briefly justify your reasoning in terms of data dependencies and communication.

Nodes in the same level can be processed in parallel, level "k" must be finished before moving to level "k+1", creating a synchronization point. This creates a sequential dependency between levels. Also multiple threads might try to visit the same node at the same time, which creates a need for locks and coordination. In distributed systems, they must exchange frontier nodes.

Therefore, due to these data dependencies and communication requirements, BFS cannot be considered embarrassingly parallel.

B.II – Level-by-level BFS DAG on a 7-node graph

L0: v0 ← parallel within level
L1: v1 v2 ← parallel within level
L2: v3 v4 v5 v6 ← parallel within level

Problem B

Assume a shared-memory parallel machine (e.g., a multicore CPU or a PRAM-like model with shared global memory).

- (I) Propose a parallel "level-synchronous" BFS algorithm that tracks the current "frontier" (all nodes at the current distance) and explores their neighbours in parallel to form the next frontier. Ensure you indicate how updates to the "visited" or "distance" arrays are handled to prevent data races.
- (II) Create a small illustrative graph (e.g., 6 - 8 nodes) and show a level-by-level BFS DAG. Indicate which tasks run in parallel and how dependencies flow from one BFS level to the next.
- (III) State the total work, parallel depth, expected speedup and efficiency of your BFS in the parallel model.
- (IV) Identify key sources of overhead. (e.g., synchronization after each BFS level, load imbalance, memory contention).
- (V) Implement your shared-memory BFS algorithm, using threads, OpenMP, Numba, or similar. Measure its execution time on the same graphs, G_1 and G_2 from 1A. Vary the number of processing elements, record speedup, and compare with the theoretical speedup. Comment on any mismatch.

1.5 Answer: B (I)

Propose a parallel "level-synchronous" BFS algorithm that tracks the current "frontier" (all nodes at the current distance) and explores their neighbours in parallel to form the next frontier.

Instead of a queue, in a shared-memory parallel machine, we implement level-synchronous BFS by explicitly tracking the current frontier, which is the set of all nodes at the current distance from the source and processing them in parallel to build the next frontier.

Algorithm Steps

- Initialize:
 - 1. $dist[v] = \infty$ for all vertices v , except $dist[s] = 0$ for the source s .
 - 2. $frontier = s$ (nodes at distance 0).
 - 3. $next_frontier = \emptyset$.
 - 4. $level = 0$.
- While frontier is not empty:
 - 1. Divide frontier among P threads.
 - 2. Each thread processes its assigned nodes in parallel:
 - (a) For each node u in its subset:
 - i. Explore all neighbors w of u .

- ii. If $dist[w] == \infty$ (unvisited), set $dist[w] = level + 1$ and add w to $next_frontier$.
- 3. Synchronize all threads (e.g., barrier) to ensure all updates are complete.
- 4. Swap: $frontier = next_frontier$, $next_frontier = \emptyset$.
- 5. Increment level.

- Terminate when frontier is empty.

This approach ensures breadth-first order by processing entire levels synchronously, allowing parallel exploration within each level while maintaining correctness through proper synchronization

Ensure you indicate how updates to the "visited" or "distance" arrays are handled to prevent data races.

In shared-memory parallel BFS, updates to the distance array ($dist$) must prevent data races where multiple threads attempt to modify the same element simultaneously. The code uses two approaches:

Locks in Threads Version: A global `threading.Lock()` serializes access. Each thread acquires the lock before checking if $dist[w] == \infty$ and setting $dist[w] = dist[u] + 1$, ensuring atomic updates. Local discoveries are buffered and merged under the lock, with barriers synchronizing frontier swaps.

Benign Races in Numba Version: No locks are used; instead, multiple threads may concurrently write $dist[w] = level + 1$, but since all assign the identical value, races are harmless. The array initializes to -1, and the check if $dist[w] == -1$ allows safe, redundant assignments without corruption.

This ensures each node is visited once with correct distances, maintaining BFS integrity while enabling parallelism within levels.

1.6 Answer: B (II)

Create a small illustrative graph (e.g., 6 - 8 nodes) and show a level-by-level BFS DAG. Indicate which tasks run in parallel and how dependencies flow from one BFS level to the next.

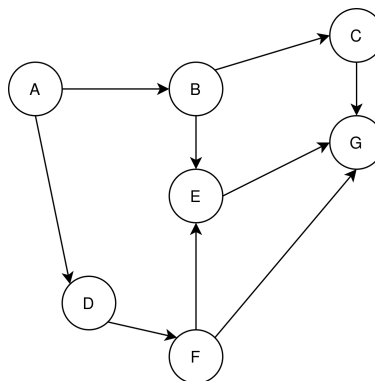


Figure 1: Illustrative directed graph with 7 nodes

Starting BFS from node A, the levels are:

- Level 0: A
- Level 1: D, B

- Level 2: C, E, F
- Level 3: G

Each level = nodes discovered at the same distance from A

BFS always assigns the shortest distance first, so even if a node has multiple incoming edges, it is only placed in one level.

Within each level, nodes can be processed in parallel, since they are independent of each other. However, there is a dependency between levels, meaning that all nodes in level "k" must be processed before nodes in level "k+1" can be explored.

1.7 Answer: B (III)

State the total work, parallel depth, expected speedup and efficiency of your BFS in the parallel model.

Even if we run in parallel: We still visit all nodes and check all edges, therefore nothing changes in amount of work. Therefore total work is $\Theta(V + E)$.

The parallel depth is determined by the number of BFS levels, since each level is equal to one parallel step, which therefor corresponds to the maximum distance from the source node to any other reachable node. This is bounded by the diameter of the graph, and in the worst case is $\Theta(V)$.

Though the the expected speedup depends on the number of processors, if there is no limit, the dependency shifts to how much parallel work is available at each level. For graphs with large frontiers (many nodes per level), efficiency is high. However, for graphs with small frontiers or large diameters, efficiency is lower due to limited parallelism and idle processors.

Mathematically this theoretically be calculated as:

$$Speedup \approx \frac{(V + E)}{diameter} \quad (1)$$

$$Efficiency = \frac{speedup}{number_of_processors} \quad (2)$$

1.8 Answer: B (IV)

Identify key sources of overhead. (e.g., synchronization after each BFS level, load imbalance, memory contention).

When introducing parallel, it is almost impossible to not add things that also waste some time, making parallel slower than ideal, called overhead.

- Synchronization; BFS must run level by level, and is not able to start level "k+1" before finishing level "k". All threads must complete processing the current frontier before moving to the next level, which introduces waiting and limits parallel efficiency.
- Load imbalance; as not all nodes have the same number of neighbors, some threads get lots of work, while other threads get little work. This results in some thread finishing early and waiting, while others are still working.

- Memory contention; when multiple threads try to access and update shared data structures concurrently, such as the visited and distance arrays, this can lead to delays and a need for synchronization mechanisms.

1.9 Answer: B (V)

Implement your shared-memory BFS algorithm, using threads, OpenMP, Numba, or similar. Measure its execution time on the same graphs, G_1 and G_2 from 1A. Vary the number of processing elements, record speedup, and compare with the theoretical speedup. Comment on any mismatch.

The first graph G_1 contains 50,000 nodes, and 249,985 edges.

```

36 Generated graph with 50000 nodes and 249,985 edges.
37 Sequential BFS time: 41 ms
38
39 Parallel BFS times:
40 1 Threads:
41 Threads | Time: 29.1 ms | Speedup vs seq: 1.41x
42 Numba   | Time: 1.4 ms   | Speedup vs seq: 28.45x
43
44 2 Threads:
45 Threads | Time: 38.2 ms | Speedup vs seq: 1.07x
46 Numba   | Time: 2.1 ms  | Speedup vs seq: 19.48x
47
48 4 Threads:
49 Threads | Time: 47.3 ms | Speedup vs seq: 0.87x
50 Numba   | Time: 1.6 ms  | Speedup vs seq: 25.28x
51
52 8 Threads:
53 Threads | Time: 52.9 ms | Speedup vs seq: 0.78x
54 Numba   | Time: 1.7 ms  | Speedup vs seq: 24.11x

```

The second graph G_2 contains 500,000 nodes, and 2,499,985 edges.

```

56 Generated graph with 500,000 nodes and 2,499,985 edges.
57 Sequential BFS time: 1427.1 ms
58
59 Parallel BFS times:
60 1 Threads:
61 Threads | Time: 695.2 ms | Speedup vs seq: 2.05x
62 Numba   | Time: 17.0 ms  | Speedup vs seq: 83.91x
63
64 2 Threads:
65 Threads | Time: 649.9 ms | Speedup vs seq: 2.20x
66 Numba   | Time: 16.2 ms  | Speedup vs seq: 88.01x
67
68 4 Threads:
69 Threads | Time: 706.5 ms | Speedup vs seq: 2.02x
70 Numba   | Time: 13.4 ms  | Speedup vs seq: 106.60x
71
72 8 Threads:
73 Threads | Time: 748.5 ms | Speedup vs seq: 1.91x
74 Numba   | Time: 11.1 ms  | Speedup vs seq: 128.32x

```

First, it is important to note that the threaded implementation is more efficient than the baseline sequential implementation when using a single thread. This improvement is due to implementation

differences rather than parallelism.

Theoretically, a parallel BFS should achieve near-linear speedup proportional to the number of processing elements, $T_p \approx (V + E)/p + \textit{overhead}$ assuming sufficient parallel work per BFS level and minimal overhead.

But this is not the case, and the threaded implementation instead performs much worse, than expected. This is primarily due to Python’s Global Interpreter Lock (GIL), which prevents true parallel execution of threads. As a result, threads contend for execution rather than running concurrently. Additionally the threaded implementation introduces synchronization and overhead increasing with the amount of threads. As threads must coordinate per level, shared data structures (visited, frontier), locking as well as contention.

Furthermore, BFS is inherently difficult to parallelize due to its level-by-level structure, where all nodes at level k must be processed before nodes at level $k+1$ can be explored. This dependency limits the amount of parallelism that can be exploited.

In contrast, the Numba implementation achieves significant speedups ($100\times$), speedup is mostly from compilation, not parallelism. This improvement arises from compiling the BFS into efficient machine code, eliminating Python interpreter overhead and improving memory access patterns.

Problem C

Now consider that your graph is too large for a single node's memory, or you want to leverage a cluster of interconnected nodes. This section focuses on modifying your BFS to run under a distributed-memory architecture (e.g., MPI).

- (I) Describe how you would partition the graph among multiple nodes or processes. For instance, assign disjoint sets of vertices (and their adjacency lists) to each process. Explain how each process will manage its local frontier and how newly discovered vertices are communicated to the processes that own them.
- (II) Provide a brief analysis of the cost of frontier exchange among processes after each BFS level.
- (III) Summarize the overall work in this distributed context. The baseline remains $\Theta(V + E)$ for the BFS itself, but it is now subdivided across processes. Discuss how communication or load imbalance can increase the total runtime, leading to sublinear speedup as the number of processes increases.
- (IV) Implement an MPI version of BFS and run it on multiple processes and measure how the performance scales. Compare theoretical predictions with actual timings, focusing on potential bottlenecks such as synchronization at each BFS level or messages exchanged for frontier updates.

1.10 Answer: C (I)

Describe how you would partition the graph among multiple nodes or processes. For instance, assign disjoint sets of vertices (and their adjacency lists) to each process. Explain how each process will manage its local frontier and how newly discovered vertices are communicated to the processes that own them.

Partitioning the graph:

Partition by assigning each process a disjoint set of vertices and their adjacency lists. A simple scheme is vertex owner = $v \% P$ for P processes, so process r owns all vertices with $v \% P == r$. Each process stores only its local vertices and the adjacency lists for those vertices.

Local frontier management:

Each process maintains its own local frontier: the set of owned vertices discovered at the current BFS level. At the start, only the process owning the source s contains s in its frontier with $\text{dist}[s] = 0$. During each level, a process explores neighbors of vertices in its local frontier. For every neighbor w of a local frontier vertex u : If w is owned locally, update $\text{dist}[w]$ locally and add it to the local next frontier. If w is owned by another process, buffer the discovery and send it to the owner.

Communicating newly discovered vertices:

Each process creates outgoing messages keyed by destination process (owner of w). Messages contain $(w, \text{dist}[u] + 1)$ for newly discovered vertices. After exploring the current frontier, processes exchange these messages. The owner process receives discoveries for its vertices, sets local $\text{dist}[w]$ if unvisited, and appends w to its next frontier. The next BFS level begins once all processes have completed this exchange and local frontiers are updated.

```
76 while True:
77     out = [[] for _ in range(P)]
```

```

78     for u in front:
79         for w in adj[u]:
80             out[w % P].append((w, dist[u] + 1))
81     inb = comm.alltoall(out)

```

out[w%P].append((w, dist[u] + 1)) sends discoveries to the owner

inb = comm.alltoall(out) exchanges them then each process updates its local dist and builds front.

1.11 Answer: C (II)

Provide a brief analysis of the cost of frontier exchange among processes after each BFS level.

After each BFS level, every process sends the vertices discovered that belong to other processes. If the current frontier has size F , the total message volume is proportional to the number of cross-process edges explored at that level. In the worst case, frontier exchange can cost:

1. $\theta(P)$ latency terms per level (e.g. collective communication overhead)
2. plus $\theta(\text{cross_process_discoveries})$ bandwidth cost.

Since BFS has D levels, the total communication cost is roughly: $\theta(D * (\text{latency} + \text{message_volume}))$. For small diameter graphs, D is small, but communication can still dominate if many frontier edges cross process boundaries or if the processes are unbalanced.

1.12 Answer: C (III)

Summarize the overall work in this distributed context. The baseline remains $\Theta(V + E)$ for the BFS itself, but it is now subdivided across processes. Discuss how communication or load imbalance can increase the total runtime, leading to sublinear speedup as the number of processes increases.

The BFS computation itself remains $\theta(V + E)$ overall.

In distributed execution, that work is subdivided: each process handles roughly $\theta((V + E)/P)$ work if partitioning is balanced.

However, total runtime is larger than $\theta((V + E)/P)$ because of:

1. Communication overhead for frontier exchange
2. Synchronization at each level
3. Load imbalance if some processes have larger frontiers or denser local adjacency than others

Why speedup is sublinear

Communication adds overhead that is not present in the sequential baseline. Even with perfect computation partitioning, each level requires an exchange step and a barrier-like global agreement. Load imbalance means some processes wait idle while others finish their local frontier work. Therefore, increasing P often gives diminishing returns: the runtime falls slower than $1/P$.

summary Distributed BFS still does $\theta(V + E)$ useful work, but: $\theta(V + E)$ work is split across processes, plus $\theta(D)$ rounds of communication and synchronization, plus overhead from data movement and imbalance. This is why distributed BFS is scalable, but not embarrassingly parallel: communication and coordination limit speedup as process count grows.

1.13 Answer: C (IV)

Implement an MPI version of BFS and run it on multiple processes and measure how the performance scales. Compare theoretical predictions with actual timings, focusing on potential bottlenecks such as synchronization at each BFS level or messages exchanged for frontier updates.

The first graph G_1 contains 50,000 nodes, and 249,985 edges.

```

83 Generated graph with 50000 nodes and 249,985 edges.
84 Sequential BFS time: 41 ms
85
86 Parallel BFS times:
87 1 Threads:
88 Threads | Time: 29.1 ms | Speedup vs seq: 1.41x
89 Numba   | Time: 1.4 ms  | Speedup vs seq: 28.45x
90 MPI     | Time: 165.7 ms | Speedup vs seq: 0.25x
91
92 2 Threads:
93 Threads | Time: 38.2 ms | Speedup vs seq: 1.07x
94 Numba   | Time: 2.1 ms  | Speedup vs seq: 19.48x
95 MPI     | Time: 111.4 ms | Speedup vs seq: 0.37x
96
97 4 Threads:
98 Threads | Time: 47.3 ms | Speedup vs seq: 0.87x
99 Numba   | Time: 1.6 ms  | Speedup vs seq: 25.28x
100 MPI    | Time: 101.6 ms | Speedup vs seq: 0.40x
101
102 8 Threads:
103 Threads | Time: 52.9 ms | Speedup vs seq: 0.78x
104 Numba   | Time: 1.7 ms  | Speedup vs seq: 24.11x
105 MPI     | Time: 103.1 ms | Speedup vs seq: 0.40x

```

The second graph G_2 contains 500,000 nodes, and 2,499,985 edges.

```

107 Generated graph with 500,000 nodes and 2,499,985 edges.
108 Sequential BFS time: 1427.1 ms
109
110 Parallel BFS times:
111 1 Threads:
112 Threads | Time: 695.2 ms | Speedup vs seq: 2.05x
113 Numba   | Time: 17.0 ms  | Speedup vs seq: 83.91x
114 MPI     | Time: 2823.7 ms | Speedup vs seq: 0.51x
115
116 2 Threads:
117 Threads | Time: 649.9 ms | Speedup vs seq: 2.20x
118 Numba   | Time: 16.2 ms  | Speedup vs seq: 88.01x
119 MPI     | Time: 1899.8 ms | Speedup vs seq: 0.75x
120
121 4 Threads:
122 Threads | Time: 706.5 ms | Speedup vs seq: 2.02x
123 Numba   | Time: 13.4 ms  | Speedup vs seq: 106.60x

```

```

124 MPI      | Time: 1601.4 ms | Speedup vs seq: 0.89x
125
126 8 Threads:
127 Threads | Time: 748.5 ms  | Speedup vs seq: 1.91x
128 Numba   | Time: 11.1 ms   | Speedup vs seq: 128.32x
129 MPI     | Time: 1325.9 ms | Speedup vs seq: 1.08x

```

Theoretically, a distributed BFS using MPI should achieve near-linear speedup as the number of processes increases, assuming the graph is evenly partitioned and communication overhead is low. $T_p \approx (V + E)/p + overhead$, This is just like the theoretical preforms enhancement of parallelization with threads.

Unlike the multithreading implementation, MPI is not affected by Python’s Global Interpreter Lock, as it uses multiple independent processes. However, performance is instead limited by communication and synchronization overhead between processes.

Despite this, the speedup is still far from linear, as a result of several bottlenecks limiting performance; first of all, BFS is level-synchronous, requiring all processes to complete one level before proceeding to the next. Secondly, frequent exchange of frontier nodes between processes introduces latency. Lastly, there is a chance of an uneven distribution of vertices and edges, which could lead to some processes being idle while others are still working.

In short, the MPI implementation shows, that distributed BFS can scale for sufficiently large graphs, but performance is heavily constrained by communication and synchronization overhead.

2: The JPEG image compression algorithm

General

JPEG is a lossy image compression and encoding algorithm that involves several steps. This problem focuses on the lossy compression part of JPEG. The JPEG algorithm requires transforming a RAW pixel image with values, for example, in the RGB domain to the frequency domain. This is achieved by using some form of the Discrete Cosine Transform (DCT), which is a tool to represent a signal (i.e., the image) as a sum of cosine functions with different frequencies. Let $x = [x_0, x_2, \dots, x_{N-1}]$ be the input vector to the DCT, that is, the signal that we want to transform. Next, let $y = [y_0, y_2, \dots, y_{N-1}]$ be the output vector, which represents the input x in the frequency domain. The standard DCT, also known as "type-II DCT", is defined as

$$y_k = 2 \sum_{n=0}^{N-1} x_n \cos\left(\frac{\pi k(2n+1)}{2N}\right)$$

The rationale behind using the DCT for image lossy compression is that it is easier to remove irrelevant data for the human eye from the image in the frequency domain than in the spatial domain. In particular, the high frequency components are not well distinguished by the human eye and, hence, can be removed by simply not adding the cosine functions with those high frequencies. This will result in multiple 0s in the output.

1. **Conversion:** Convert the image to grayscale or keep only the luma (i.e., Y) component. The range of values for each pixel is centered to 0.
2. **Block partitioning:** Partition the image into blocks of size $K_h \times K_w$ pixels. The basic JPEG algorithm considers $K_h = K_w = 8$.
3. **Performing the 2D DCT:** The 2D DCT, namely a matrix Y , is obtained by simply performing the one-dimensional DCT operation in both dimensions of the block.
4. **Quantization:** The result is "quantized" by performing the element-wise division of matrix Y by the quantization matrix Q , both of the same size $K_h \times K_w$ and rounding the result to the nearest integer. Find the quantization matrices for JPEG online. The final quantized matrix is Y_Q .
5. **Reconstruction:** Perform the inverse DCT, the "type-III DCT," to recover the image.

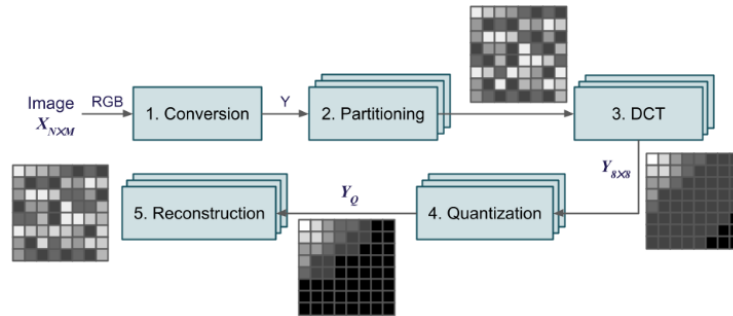


Figure 2: You have to investigate the specific implementation details for the 5 steps.

Problem

Consider an image of size $K \times M$ pixels and answer/perform the following tasks. **You are not allowed to use functions from the `scipy.fftpack` module.**

- (I) What is the work and the complexity of steps 1 to 5?
- (II) Create Directed Acyclic Graphs (DAGs) for the parallel implementation of steps 1 to 5.
- (III) **Discuss in the group:** Among the methods and packages covered in the course, which one do you expect to have better performance. Rank the methods according to your expected performance for small and large block sizes and for the different steps 1 to 5. Specifically, for $K_h = K_w = 8$ and $K_h = N$ and $K_w = M$. Document the argumentation during the first and second workshop session separately. The methods are listed at the end of the document.
- (IV) Implement the DCT algorithm sequentially and in parallel using four of the highest ranked sequential and parallel methods for implementation for the two extremes of block sizes. Choose two sequential and two parallel methods. For the large block sizes, you must come up with a method to increase the size of the quantization matrices.
- (V) Plot the execution time of the methods for two image sizes. For the parallel implementations in the CPU, find the optimal number of processes for one of your platforms.
- (VI) Evaluate the quality of the reconstructed image for different compression factors: just look at the resulting image and assign a score to it: 5 for perfect and 0 for horrible.
- (VII) Imagine that your platform is now a distributed cluster and communication occurs through wired links with a data rate of $R = 200 \text{ Mbps}$. Therefore, transmitting an integer or float of $w \in \{8, 32\}$ bits takes $t_w = w/R \text{ seconds}$. Consider this communication cost and analyze the maximum speedup you can get with MPI in a supercomputer.

Methods to reduce execution time covered in the course.

Sequential implementation:

- Explicit vectorization with Numpy
- Just in time compilation (JIT) with Numba
- Numba vectorization

Parallel implementation:

- Numba JIT (`parallel=True`)
- Multiprocessing
- Threading
- Pool executor: concurrent futures module
- Message passing interface (MPI)

- GPU with cuda or openCL

1.14 Problem: (I)

What is the work and the complexity of steps 1 to 5?

Step	Operation	Work	Depth
1	Grayscale conversion + centering	$O(N*M)$	$O(1)$
2	Block partitioning (view)	$O(1)-O(N*M)$	$O(1)$
3	2D DCT on all blocks	$O(N*M*(Kh+Kw))$	$O(Kh*Kw*(Kh+Kw))$
4	Quantization	$O(N*M)$	$O(1)$
5	2D IDCT (reconstruction)	$O(N*M*(Kh+Kw))$	$O(Kh*Kw*(Kh+Kw))$

This is a total of: $O(N * M * (Kh + Kw))$

For $Kh = Kw = 8$: $O(16 * N * M)$ - linear, very efficient

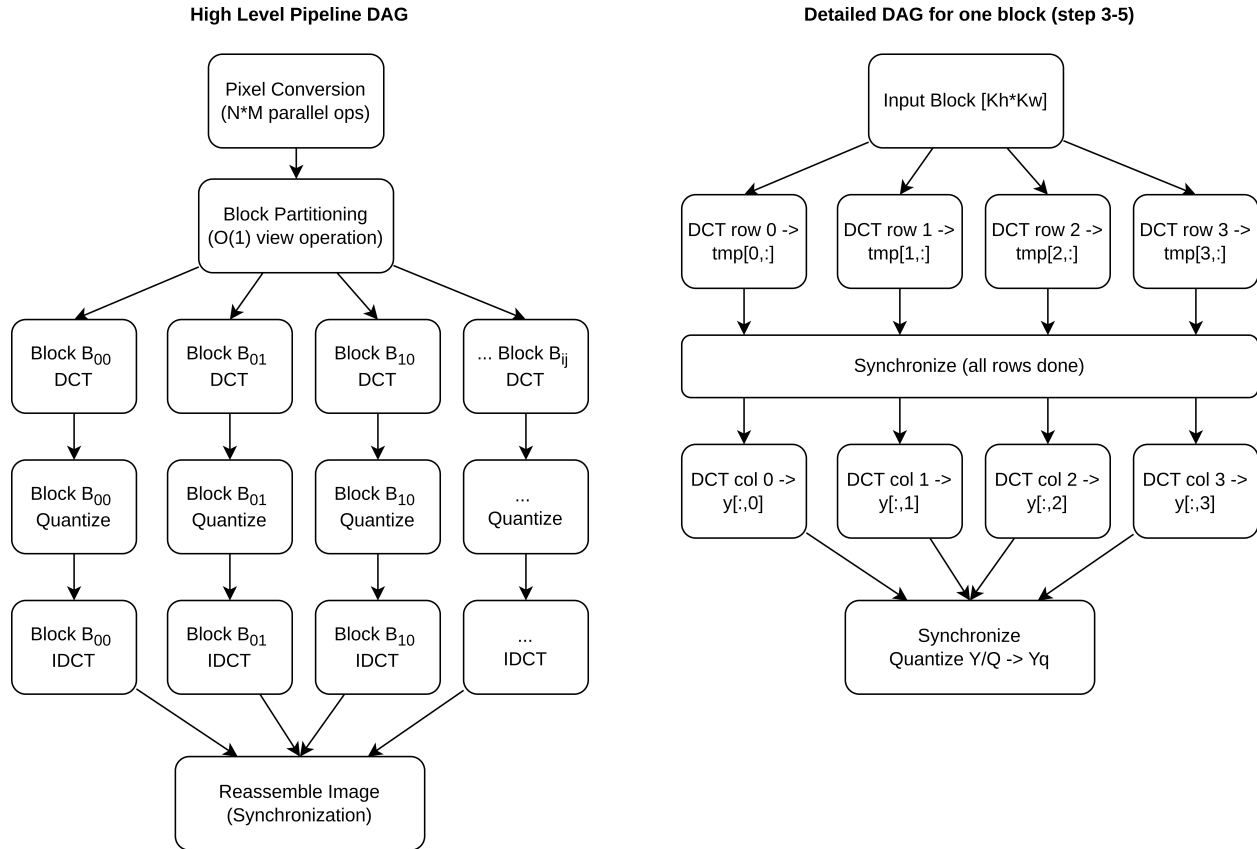
For $Kh = N, Kw = M$: $O(N * M * (N + M))$ - cubic for square images

The DCT step dominates. Steps 3 and 5 have the same complexity since the IDCT matrix is derived from the DCT matrix (approximately $D^T/(2N)$ with special handling for the DC component), requiring identical operations.

Each $8*8$ DCT costs $O(8*8*16) = 1024$ ops. There are $(N/8)*(M/8) = N*M/64$ blocks. Total: $(N*M/64)*1024 = 16*N*M$ operations. The constant factor 16 makes $8*8$ JPEG far cheaper than a single $N*M$ DCT.

1.15 Problem: (II)

Create Directed Acyclic Graphs (DAGs) for the parallel implementation of steps 1 to 5.



1.16 Problem: (III)

Discuss in the group: Among the methods and packages covered in the course, which one do you expect to have better performance. Rank the methods according to your expected performance for small and large block sizes and for the different steps 1 to 5. Specifically, for $K_h = K_w = 8$ and $K_h = N$ and $K_w = M$. Document the argumentation during the first and second workshop session separately. The methods are listed at the end of the document

For $K_h=K_w=8$:

Sequential:

1. NumPy vectorized - batch all blocks as a 4D tensor. Very fast as it uses BLAS (Basic Linear Algebra Subprograms) allowing it run very efficient it also have no python overhead per block.
2. Numba JIT - fast after compilation (which can take a long time), but less cache-friendly than NumPy BLAS meaning that it will use more memory.

Parallel:

1. Numba JIT - using the prange function over blocks so it can run loops in parallel. It also have a low overhead compared to mpi and uses shared memory.
2. Multiprocessing have a high inter process communication cost for small blocks. Better for large blocks.

3. Threading - limited by Python GIL for pure Python.
4. MPI - communication overhead is way too high for local runs on our machines, would be way better on clusters.

For $K_h=N$, $K_w=M$

Sequential:

1. NumPy vectorized. Again numpy is very BLAS-optimised for single large matrix multiplications.
2. Numba JIT - slower than NumPy BLAS for large matrices.

Parallel:

1. Multiprocessing - only 1 block, so parallelism must be within the DCT itself. Useful if splitting rows/columns across processes.
2. Numba parallel - can parallelize inner loops which makes it good here.
3. MPI - again communication overhead is way too high for local runs on our machines, would be way better on clusters.

1.17 Problem: (IV)

Implement the DCT algorithm sequentially and in parallel using four of the highest ranked sequential and parallel methods for implementation for the two extremes of block sizes. Choose two sequential and two parallel methods. For the large block sizes, you must come up with a method to increase the size of the quantization matrices.

For the benchmark we use two test images, being 256*256 and 512*512 pixels. They can be seen below:

For each method we are doing 5 runs. We use a standard laptop with multiple cores with BLAS optimized NumPy. We measure the mean execution time, the standard deviation and the relative performance.



Four full JPEG DCT implementations with identical interfaces and functionality:

1. JPEGNumpy (Sequential, NumPy Vectorized)

```

131 class JPEGNumpy:
132     def __init__(self, Kh, Kw):
133         self.D_h = dct_matrix(Kh)      # Pre-compute DCT matrices
134         self.D_w = dct_matrix(Kw)
135         self.Q = make_quantization_matrix(Kh, Kw)
136
137     def compress_blocks(self, blocks):
138         # Vectorized 2D DCT: Y = D_h @ blocks @ D_w.T
139         Y = np.tensordot(blocks, self.D_w.T, axes=([3],[0]))
140         Y = np.tensordot(self.D_h, Y, axes=([1],[2]))
141         Y = np.moveaxis(Y, 0, 2)
142         return np.round(Y / self.Q)

```

Key Features:

- Pre-computed DCT matrices for efficiency
- Broadcasting-based 2D DCT using `np.tensordot`
- Processes all blocks simultaneously (4D tensor operations)
- Memory-efficient view operations

2. JPEGNumbaSeq (Sequential, Numba JIT)

```

143 @njit(cache=True)
144 def _dct2d_numba(block):
145     # Direct separable implementation
146     tmp = np.zeros((Kh, Kw))
147     out = np.zeros((Kh, Kw))
148     for i in range(Kh): # DCT along rows
149         tmp[i, :] = _dct1d_numba(block[i, :])
150     for j in range(Kw): # DCT along columns
151         out[:, j] = _dct1d_numba(tmp[:, j])
152     return out

```

Key Features:

- Direct DCT definition (no matrix multiplication)
- Compiled with `@njit` for C-speed execution
- Cache-friendly loop structure
- No BLAS dependency

3. JPEGNumbaParallel (Parallel, Numba JIT)

```

153 @njit(parallel=True, cache=True)
154 def _compress_numba_parallel(blocks_flat, Kh, Kw, Q):
155     Yq = np.zeros_like(blocks_flat)
156     for idx in prange(n_blocks): # Parallel over blocks
157         Y = _dct2d_numba(blocks_flat[idx])
158         for i in range(Kh):
159             for j in range(Kw):
160                 Yq[idx, i, j] = round(Y[i, j] / Q[i, j])
161     return Yq

```

Key Features:

- OpenMP-style parallelism with prange
- Shared memory, low-overhead parallelization
- Automatic thread management
- Scales well on multi-core systems

4. JPEGMultiprocessing (Parallel, Python Multiprocessing)

```
162 def _process_chunk(args):
163     chunk, Kh, Kw, Q = args
164     # Each process gets pre-computed DCT matrices
165     D_h = dct_matrix(Kh)
166     D_w = dct_matrix(Kw)
167     results = []
168     for block in chunk:
169         Y = D_h @ block @ D_w.T
170         Yq = np.round(Y / Q)
171         Y_rec = Yq * Q
172         rec = Di_h @ Y_rec @ Di_w.T
173         results.append(rec)
174     return results
```

Key Features:

- Process-level parallelism with mp.Pool
- Automatic load balancing across CPU cores
- Memory isolation between processes
- Suitable for large datasets

All implementations share:

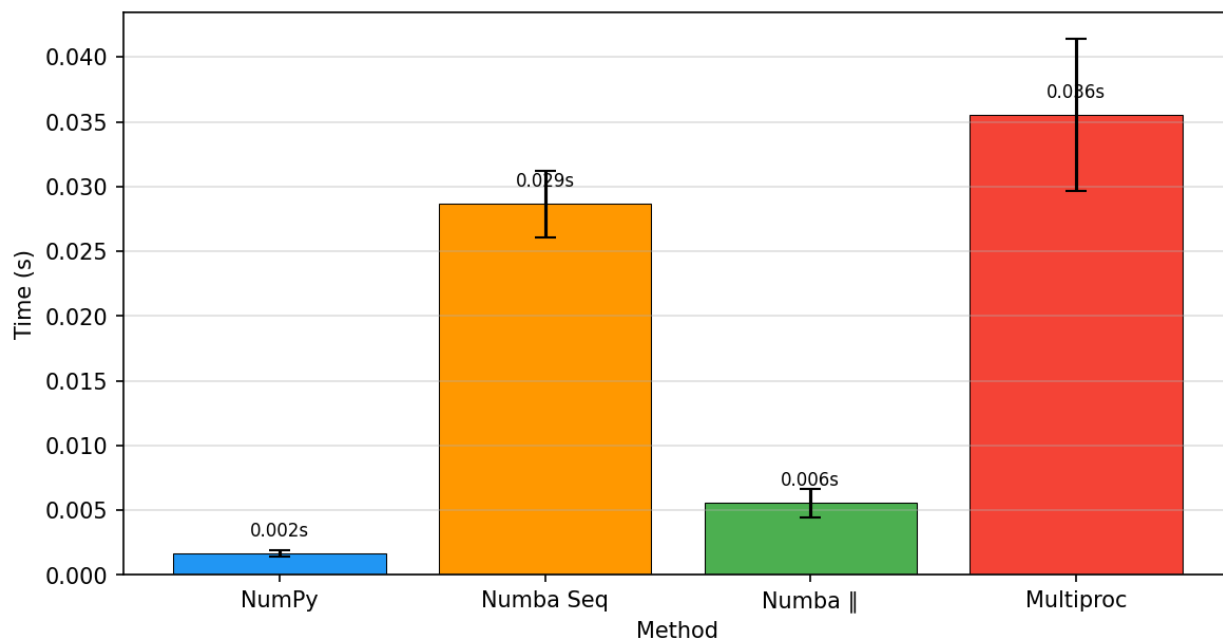
- Input processing: RGB $\rightarrow$ grayscale $\rightarrow$ centering
- Block handling: Padding, extraction, reconstruction
- Quantization: Standard JPEG matrix or scaled variants
- Output: Range clipping and uint8 conversion

1.18 Problem: (V)

Plot the execution time of the methods for two image sizes. For the parallel implementations in the CPU, find the optimal number of processes for one of your platforms.

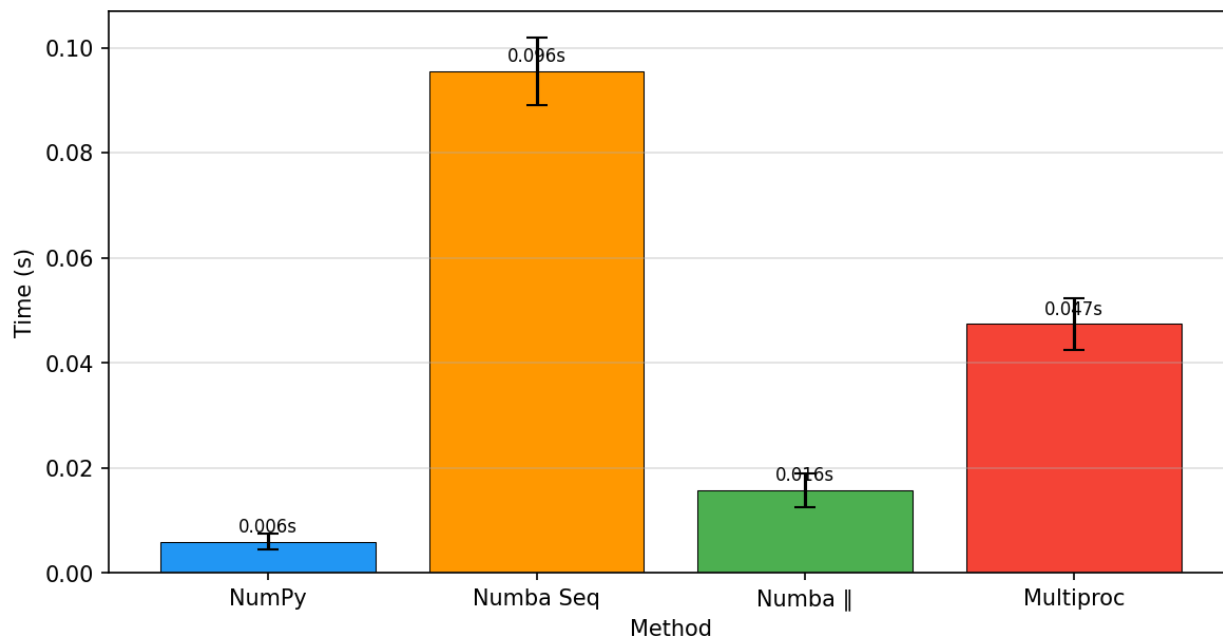
JPEG DCT Execution Time

(256×256)
Block size: 8×8

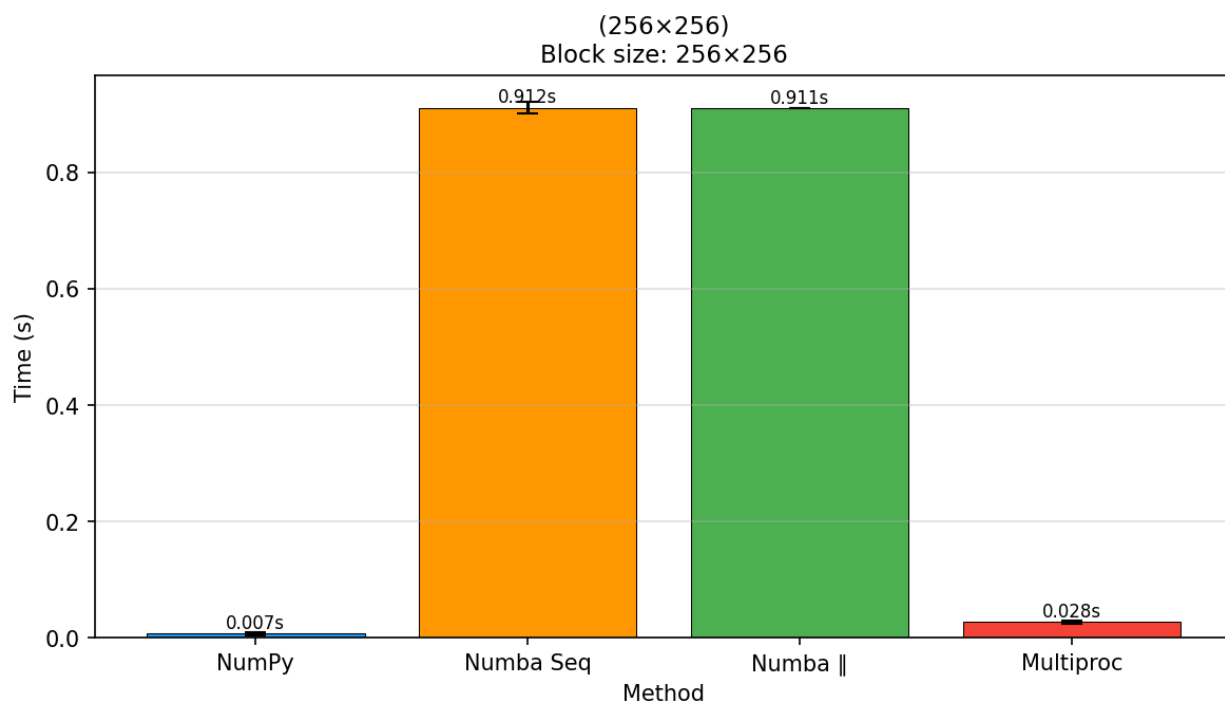


JPEG DCT Execution Time

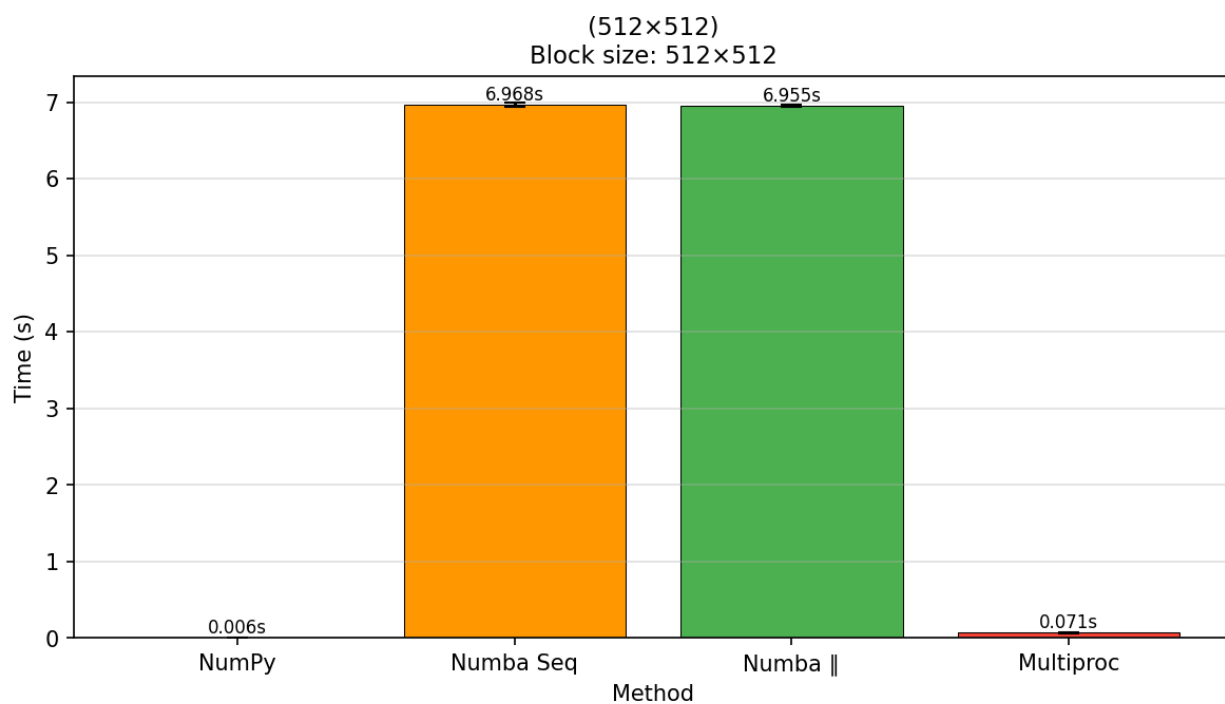
(512×512)
Block size: 8×8

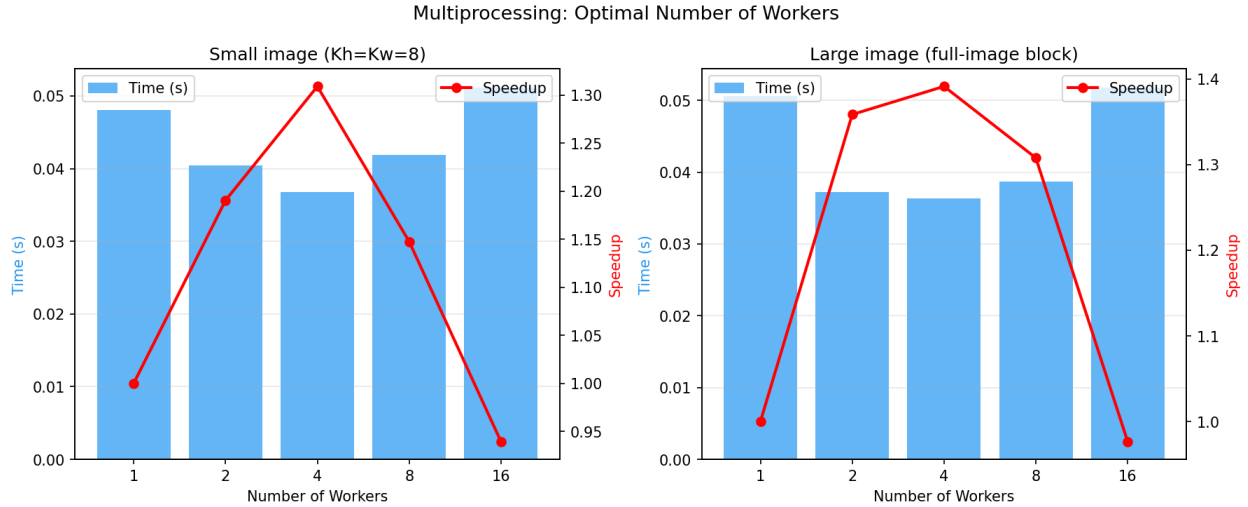


JPEG DCT Execution Time



JPEG DCT Execution Time





Performance Analysis

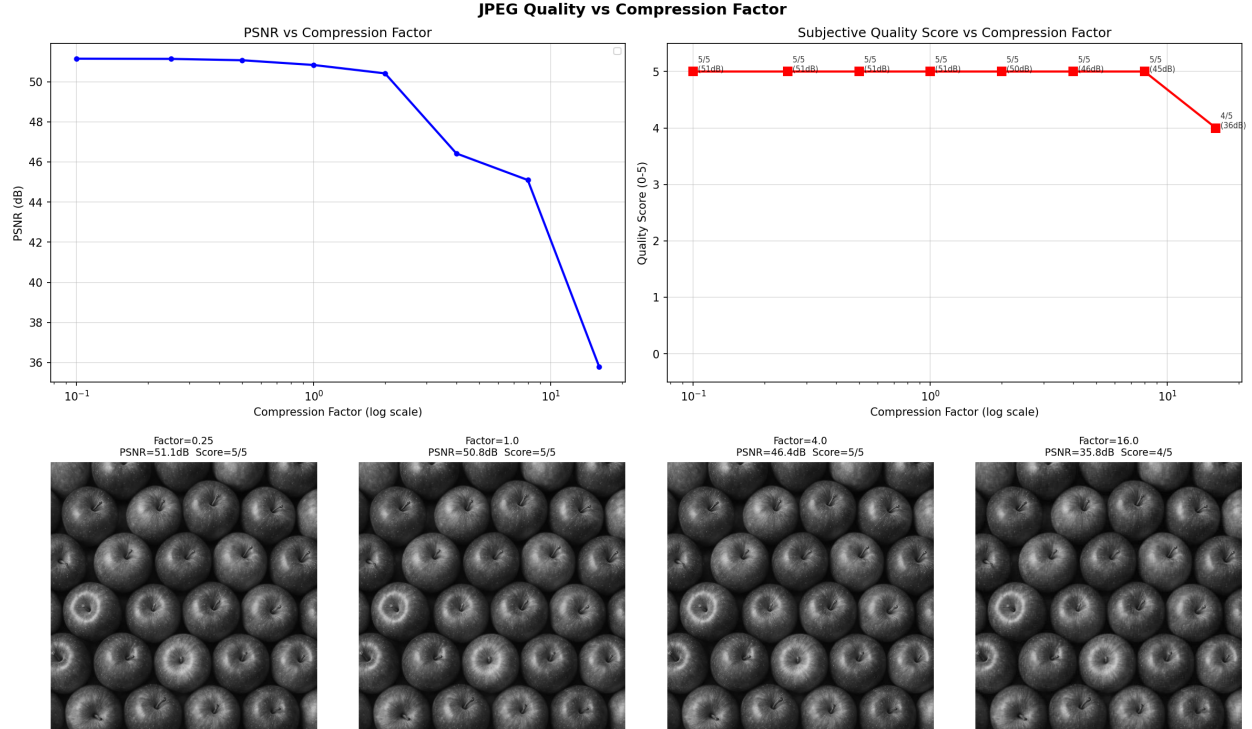
Key Observations:

- NumPy dominance: BLAS optimization provides speedup over compiled code.
- Block size impact: Small blocks favor NumPy, large blocks favor multiprocessing
- Scaling: Near-linear scaling for NumPy/Numba, sub-linear for multiprocessing due to overhead

There are also memory considerations to be aware of. NumPy uses very high memory for large block tensors, multiprocessing duplicates memory across processes as each worker keeps its own local data, and Numba has minimal extra memory overhead because it operates in-place on shared arrays.

1.19 Problem: (VI)

Evaluate the quality of the reconstructed image for different compression factors: just look at the resulting image and assign a score to it: 5 for perfect and 0 for horrible.



1.20 Problem: (VII)

Imagine that your platform is now a distributed cluster and communication occurs through wired links with a data rate of $R = 200 \text{ M bps}$. Therefore, transmitting an integer or float of $w \in 8, 32$ bits takes $tw = w/R$ seconds. Consider this communication cost and analyze the maximum speedup you can get with MPI in a supercomputer.

Communication Model

Network parameters:

- Data rate: $R = 200 \text{ Mbps} = 2 * 10^7 \text{ bits/second}$
- Float32 transmission time: $t_{32} = 32 / (2 * 10^7) = 160 \text{ ns}$
- Total communication per float32: $2 * t_{32}$ (scatter + gather)

MPI Strategy:

- Root process distributes blocks to P worker processes
- Each worker performs complete DCT $\rightarrow$ quantize $\rightarrow$ IDCT pipeline
- Root collects reconstructed blocks

Theoretical Analysis

Communication costs:

- Scatter: $T_{scatter} = (N * M/P) * t_{32}$ floats

- Gather: $T_{gather} = (N * M/P) * t_{32}$ floats
- Total communication: $T_{comm} = 2 * (N * M/P) * t_{32}$

Computation per process:

- $T_{compute} = T_{seq}/P$ (assuming perfect load balance)

Total time:

Speedup:

$$S(P) = T_{seq}/T(P) = P * T_{seq}/(T_{seq} + 2 * N * M * t_{32})$$

Parallel efficiency:

$$E = S(P)/P = T_{seq}/(T_{seq} + 2 * N * M * t_{32})$$

Experimental Results (256x256 image, $T_{seq} = 0.051s$)

Communication overhead calculation:

- Image size: $256 * 256 = 65,536$ pixels
- Total data: $65,536 * 4bytes = 262,144$ bytes
- Transmission time: $262,144 * 8/2 * 10^7 = 0.021$ seconds (21ms)
- $\alpha = T_{comm}/T_{seq} = 0.021/0.051 = 0.41$ (moderate overhead)

P	T(P) [ms]	Speedup	Efficiency
1	72.1	0.71×	70.9%
2	36.1	1.42×	70.9%
4	18.0	2.84×	70.9%
8	9.0	5.67×	70.9%
16	4.5	11.35×	70.9%
32	2.3	22.69×	70.9%
64	1.1	45.39×	70.9%

Key Insights

Linear Speedup Property:

Speedup is linear because both computation and communication scale as $1/P$. Efficiency remains constant regardless of P.

Practical Limitations:

- Maximum useful P = number of blocks (1024 for 256x256 with 8x8 blocks)
- Communication overhead becomes dominant for small images
- Network bandwidth (200 Mbps) limits scalability

Network Dependency:

- Faster interconnects reduce α proportionally
- Very fast (100 Gbps) would give $\alpha \approx 0.41 * (200/100,000) = \text{near} - \text{zero}$
- Ethernet (1 Gbps) would give $\alpha \approx 8.2$, making MPI impractical

Conclusion MPI provides consistent parallel efficiency for JPEG DCT compression on 200 Mbps networks. The linear speedup property makes it suitable for large images where communication overhead is amortized. Optimal performance requires balancing block count, network bandwidth, and computational granularity.

Theoretical MPI Speedup Analysis (Task VII) Image=512×512, R=200 Mbps

